

Guide general pour deployer un site web

Hebergement, nom de domaine, serveur Node.js, variables d'environnement, paiement Stripe et verifications

Auteur : Hocine Korichi - Avril 2026

Ce document fusionne les recommandations sur les hebergeurs et les etapes de transfert vers un hebergement. Le contenu est general : il peut s'appliquer a un site personnel, un site vitrine, une application Node.js ou un site avec paiement.

1. Objectif du document

Ce guide explique comment passer d'un site developpe localement a un site publie sur Internet. Il distingue les sites simples, qui peuvent etre heberges comme fichiers statiques, et les sites plus avances, qui exigent un serveur actif.

- Choisir le bon type d'hebergement selon la technologie utilisee.
- Preparer les fichiers avant le transfert.
- Configurer un serveur Node.js lorsque le site utilise une API ou un paiement.
- Configurer un nom de domaine au bon moment.
- Eviter de publier des cles secretees ou des fichiers sensibles.

2. Choisir le bon type de site

Type de site	Technologies habituelles	Hebergement recommande
Site statique simple	HTML, CSS, JavaScript, images	GitHub Pages, Cloudflare Pages, Netlify, Vercel
Site classique avec PHP	HTML, CSS, PHP, MySQL	Hebergement cPanel, Hostinger, GoDaddy, OVH
Application Node.js	server.js, Express, API, variables d'environnement	Render, Railway, Vercel, VPS, cPanel compatible Node.js
Site avec paiement	Node.js + Stripe ou PayPal	Hebergement Node.js securise, avec HTTPS et variables d'environnement

Erreur a eviter : heberger une application Node.js comme une simple page HTML. Si le site depend d'un fichier serveur, comme server.js, il faut un hebergement capable de l'executer.

3. Meilleurs hebergeurs selon le besoin

Pour un site personnel simple

Hebergeur	Avis	Utilisation conseillee
Cloudflare Pages	Excellent	Site statique rapide, gratuit, securise
Netlify	Tres bon	Site statique, formulaires simples, deploiement facile
GitHub Pages	Tres bon	Site personnel gratuit, documentation, portfolio
Vercel	Tres bon	Site moderne, JavaScript, projets evolutifs
Hostinger	Bon choix classique	Site vitrine, domaine, courriels, cPanel
GoDaddy	Correct	Site simple ou cPanel, a verifier pour Node.js

Pour un site avec serveur, API ou paiement

Hebergeur	Avis	Remarque
Render	Recommande	Simple pour Node.js, Express et variables d'environnement
Railway	Bon choix	Adapte aux projets Node.js, attention au cout selon usage
Vercel	Excellent	Tres bon, mais peut exiger une adaptation vers des fonctions serveur

Hostinger VPS	Bon choix technique	Plus de controle, mais plus d'administration
OVH VPS	Bon choix technique	Solide, mais demande des competences serveur
GoDaddy cPanel	Possible sous condition	Il faut un forfait compatible Node.js

4. Faut-il acheter un nom de domaine avant ?

Non. Il n'est pas obligatoire d'acheter un nom de domaine avant le deploiement. La plupart des plateformes fournissent une adresse temporaire gratuite pour tester le site.

Plateforme	Exemple d'adresse fournie
Render	https://nom-du-site.onrender.com
Vercel	https://nom-du-site.vercel.app
Netlify	https://nom-du-site.netlify.app
Cloudflare Pages	https://nom-du-site.pages.dev

Ordre conseille : developper en local, deployer avec l'adresse gratuite, tester, corriger, acheter le nom de domaine, le relier au site, puis passer en production.

Le nom de domaine devient utile quand le site est pret a etre publie officiellement ou quand il doit inspirer confiance.

5. Architecture generale d'un site avec serveur

Pour un site avec API, paiement ou logique serveur, l'architecture generale est la suivante :

```

Navigateur de l'utilisateur
->
Page HTML / interface
->
Serveur Node.js avec Express
->
Service externe : Stripe, PayPal, base de donnees ou API
->
Page de confirmation ou reponse JSON

```

Role des fichiers courants

Fichier ou dossier	Role
index.html	Interface principale visible par l'utilisateur.
success.html	Page affichee apres une action reussie, par exemple un paiement confirme.
server.js	Serveur Node.js : routes API, paiement, verification, traitement securise.
package.json	Liste des dependances et scripts de demarrage.
package-lock.json	Versions exactes des dependances. A transferer si disponible.
.env	Cles secretes en local seulement. Ne pas publier.
img/	Images du site.

6. Fichiers a transferer

A transferer vers l'hebergeur

- index.html
- success.html si une page de retour est utilisee
- server.js si le projet utilise Node.js
- package.json
- package-lock.json si disponible
- dossier img/ ou assets/

A ne pas transferer comme fichiers publics

- node_modules : il sera recree avec npm install.
- .env : il contient des secrets. Utiliser les variables d'environnement de l'hebergeur.
- Fichiers de sauvegarde, anciennes copies, journaux, cles privees.

7. Preparer le projet en local

Avant de transferer le site, il doit fonctionner localement.

```
cd C:\xampp\htdocs\nom-du-projet
npm install
npm start
```

Puis ouvrir dans le navigateur :

```
http://localhost:3000
```

Si le site ne fonctionne pas en local, il ne fonctionnera pas mieux chez l'hebergeur. Il faut d'abord corriger les erreurs locales.

8. Adapter server.js pour un hebergement en ligne

Sur un hebergeur, le port n'est pas forcement 3000. Il faut utiliser le port fourni par l'environnement.

Remplacer :

```
const PORT = 3000;
```

par :

```
const PORT = process.env.PORT || 3000;
```

Si le site utilise un retour apres paiement ou apres validation, remplacer les adresses localhost par le vrai domaine.

```
En local :
success_url: "http://localhost:3000/success.html"
cancel_url: "http://localhost:3000/index.html"
```

```
En ligne :
success_url: "https://ton-domaine.com/success.html"
cancel_url: "https://ton-domaine.com/index.html"
```

9. Variables d'environnement et securite

Les cles secretees ne doivent jamais etre ecrites dans index.html ni publiees dans un depot public.

En local, on peut utiliser un fichier .env :

```
STRIPE_SECRET_KEY=sk_test_TA_CLE_SECRETE_ICI
```

Chez l'hebergeur, il faut creer une variable d'environnement avec le meme nom :

```
STRIPE_SECRET_KEY
```

La cle publique peut etre visible dans une page si le service l'exige. La cle secrete, elle, doit rester uniquement cote serveur.

10. Paiement Stripe ou PayPal

Un vrai paiement exige un serveur securise. Une page HTML seule ne suffit pas. Le serveur cree la session de paiement, puis le client est redirige vers Stripe Checkout ou PayPal.

Element	Role
Page HTML	Collecte le choix de l'utilisateur et appelle le serveur.
Serveur Node.js	Cree la session de paiement avec la cle secrete.
Stripe ou PayPal	Affiche le formulaire securise de carte ou de compte.
Page de confirmation	Affiche la reference apres retour du service de paiement.

Important : Stripe confirme le paiement. Il ne cree pas automatiquement une reservation, un billet ou une prestation metier. Pour cela, il faut connecter aussi l'API du service concerne.

11. Deploiement sur Render

- Creer un compte Render.
- Creer un depot Git contenant le projet, sans fichier .env.
- Dans Render, choisir New Web Service.
- Connecter le depot du projet.
- Indiquer npm install comme commande d'installation.
- Indiquer npm start comme commande de demarrage.
- Ajouter les variables d'environnement necessaires.
- Deployer le service.
- Tester l'adresse fournie par Render.
- Remplacer les adresses de retour par l'adresse Render ou par le domaine final.

12. Deploiement sur Vercel

Vercel est excellent pour les sites modernes. Pour une application Express classique, il peut etre necessaire d'adapter le serveur en fonctions API.

- Creer un compte Vercel.
- Importer le depot Git.
- Configurer les variables d'environnement.
- Adapter les routes serveur si le projet n'est pas deja compatible.
- Deployer puis tester les routes API et les pages.

13. Deploiement sur Railway

- Creer un compte Railway.
- Creer un nouveau projet depuis un depot Git.
- Definir les variables d'environnement.
- Verifier la commande de demarrage npm start.
- Deployer, tester l'adresse fournie, puis relier un domaine si necessaire.

14. Deploiement chez GoDaddy avec cPanel

Cette option fonctionne seulement si le forfait GoDaddy propose une option compatible Node.js, souvent appelee Setup Node.js App ou equivalent.

- Ouvrir cPanel.
- Chercher l'option Node.js.
- Creer une nouvelle application Node.js.
- Choisir le dossier du projet.
- Indiquer server.js comme fichier de demarrage.
- Configurer la version de Node.js disponible.
- Definir les variables d'environnement.
- Lancer npm install depuis l'interface ou par SSH.
- Demarrer ou redemarrer l'application.
- Tester le domaine ou le sous-domaine fourni.

Si l'hebergement GoDaddy ne permet pas Node.js, l'application avec serveur ne fonctionnera pas correctement. Dans ce cas, utiliser Render, Railway, Vercel adapte, ou un VPS.

15. Relier un nom de domaine

Lorsque le site fonctionne avec l'adresse temporaire de l'hebergeur, on peut relier un nom de domaine.

- Acheter le domaine chez un registraire : GoDaddy, Namecheap, OVH, Cloudflare Registrar, etc.
- Dans l'hebergeur, ajouter le domaine personnalise.
- Dans le gestionnaire DNS du domaine, creer les enregistrements demandes par l'hebergeur.
- Attendre la propagation DNS.
- Verifier que le certificat HTTPS est actif.
- Mettre a jour les adresses de retour dans le serveur : success_url, cancel_url, liens internes.

Il est preferable de ne pas acheter le domaine trop tot. Tester d'abord le site avec l'adresse gratuite evite des frais inutiles et des reglages precoces.

16. Commandes utiles

Installation :
npm install

Demarrage :
npm start

Arret local :
CTRL + C

Redemarrage apres modification du serveur :
npm start

17. Verifications apres transfert

- La page principale s'ouvre correctement en HTTPS.
- Les images se chargent.
- Les formulaires fonctionnent.
- Les routes API retournent du JSON et non une page HTML d'erreur.
- Le bouton de paiement redirige vers Stripe ou PayPal si le site utilise un paiement.
- La page de confirmation s'affiche apres l'action reussie.
- Aucune cle secrete n'apparait dans le code HTML visible.
- Le site fonctionne aussi sur telephone.
- Le nom de domaine pointe vers le bon hebergement.
- Le certificat HTTPS est actif.

18. Erreurs frequentes

Erreur	Cause probable	Correction
ERR_CONNECTION_REFU	Serveur non lance ou mauvais port	Demarrer le serveur et verifier le port
Reponse non JSON	La route API n'existe pas ou retourne HTML	Verifier server.js et l'URL appelee
Paiement impossible	Cle secrete absente ou mauvaise	Configurer la variable d'environnement
Images absentes	Chemin incorrect ou dossier non transfere	Verifier img/ ou assets/
Site OK localement, pas en ligne	Port fixe ou variables manquantes	Utiliser process.env.PORT et configurer les variables
Domaine ne fonctionne pas	DNS non propage ou mal configure	Verifier les enregistrements DNS

19. Limites importantes

- Un hebergement statique ne peut pas executer un serveur Node.js permanent.
- Un paiement reel exige une passerelle de paiement et un serveur securise.
- Une confirmation de paiement ne remplace pas une confirmation metier : reservation, commande, billet, dossier client, etc.
- Les cles secretees doivent rester hors du code public.
- Un site professionnel doit etre teste sur ordinateur et telephone avant publication.

20. Resume final

Pour un site personnel simple, Cloudflare Pages, Netlify, GitHub Pages ou Vercel suffisent souvent. Pour un site avec serveur Node.js, API ou paiement, Render ou Railway sont generalement plus simples. GoDaddy peut convenir seulement si le forfait accepte Node.js. Le nom de domaine peut etre achete apres les premiers tests. Les cles secretees doivent rester dans les variables d'environnement, jamais dans le HTML.